

PROJECT REPORT OF CUSTOMER SUPPORT TICKET RESOLUTION PIPELINE

(A PySpark-Based ETL Pipeline using Databricks and Delta Lake)

Submitted By

Ritik Chanana

Bachelor of Technology (B.Tech)

Computer Science & Engineering

Maharishi Markandeshwar (Deemed to be University)

Submitted To

Celebal Technologies

Celebal Excellence Internship

Project Domain

Data Engineering

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the support, guidance, and encouragement of several individuals and organizations. I would like to take this opportunity to express my sincere gratitude to everyone who contributed to the successful execution of this internship project.

First and foremost, I would like to express my heartfelt appreciation to **Celebal Technologies** for providing me with the opportunity to undertake this project as part of my internship program. The internship has provided me with valuable exposure to industry-oriented data engineering practices and has allowed me to apply theoretical concepts to solve practical business problems. Working on this project has significantly enhanced my understanding of modern data processing frameworks, cloud-based analytics platforms, and ETL pipeline development.

I would also like to extend my gratitude to the mentors and technical guides at Celebal Technologies for providing continuous support, valuable suggestions, and constructive feedback throughout the project. Their guidance helped me understand the importance of designing scalable data pipelines, implementing efficient data validation techniques, and following industry-standard engineering practices. Their encouragement motivated me to overcome technical challenges and improve the overall quality of the project.

I sincerely thank the faculty members of **Maharishi Markandeshwar (Deemed to be University)** for providing me with a strong academic foundation in computer science, programming, and data analytics. The knowledge and skills gained during my coursework played an important role in understanding the concepts required to successfully complete this project.

I am also grateful to my family and friends for their constant encouragement, motivation, and unwavering support throughout the internship period. Their confidence in my abilities inspired me to remain focused and dedicated while completing the project within the given timeline.

Finally, I would like to acknowledge all the online documentation, learning resources, and official technical references that assisted me in understanding PySpark, Databricks, Delta Lake, and modern ETL architectures. These resources contributed significantly to enhancing my practical knowledge and helped me successfully implement the Customer Support Ticket Resolution Pipeline.

This internship project has been an enriching learning experience that has strengthened my technical skills, improved my problem-solving abilities, and provided valuable insights into real-world data engineering workflows. I sincerely appreciate everyone who contributed directly or indirectly towards the successful completion of this project.

ABSTRACT

The increasing volume of customer support requests in modern organizations has created a growing need for efficient data processing systems capable of transforming raw operational data into meaningful business insights. Customer support ticket datasets often contain inconsistencies such as missing values, incomplete records, invalid employee identifiers, unresolved tickets, and resolution time stored in textual formats. These issues reduce the reliability of analytical reports and make it difficult for management to accurately evaluate team performance and operational efficiency.

The primary objective of this project is to design and implement an end-to-end Extract, Transform, and Load (ETL) pipeline using **PySpark** on the **Databricks** platform. The project follows the **Medallion Architecture**, consisting of Bronze, Silver, and Gold layers, to ensure a structured and scalable approach to data processing. The Bronze layer is responsible for ingesting raw customer support data without modification. The Silver layer performs data validation, cleansing, transformation, and enrichment by removing invalid records, converting textual resolution times into numerical minute values, and applying predefined business rules. Finally, the Gold layer aggregates the processed data to generate meaningful performance reports for business analysis.

Several business rules have been implemented to ensure the quality and consistency of the final analytical outputs. These include removing records with missing Ticket IDs, Agent IDs, or Resolution Time values, considering only resolved tickets with a resolution time greater than fifteen minutes, restricting analysis to Team Leads ranging from **TL01 to TL08**, and implementing a carry-over rule that prevents duplicate counting of agents who have already contributed valid resolutions on the first operational day.

The final outputs of the pipeline include a **Team Lead Performance Matrix** and a **Per-Agent Daily Performance Matrix**, both of which provide valuable insights into customer support efficiency. These reports enable management to evaluate individual and team performance while ensuring that the generated metrics accurately reflect operational activities.

This project demonstrates the practical application of distributed data processing technologies, data quality management techniques, and business-rule-driven analytics using modern data engineering tools such as Databricks, Apache Spark, PySpark, Delta Lake, and Spark SQL. Furthermore, the implemented solution provides a scalable foundation that can be extended to cloud environments such as **Microsoft Azure Data Lake Storage (ADLS)** and **Azure Databricks**, making it suitable for real-world enterprise applications.

Keywords: *PySpark, Databricks, ETL Pipeline, Delta Lake, Apache Spark, Medallion Architecture, Data Engineering, Customer Support Analytics, Business Intelligence, Data Validation.*

CHAPTER 1

INTRODUCTION

1.1 Background

In today's digital era, customer satisfaction plays a crucial role in determining the success of any organization. Businesses across industries receive a large number of customer support requests every day regarding product issues, billing concerns, account-related problems, delivery delays, and technical assistance. These requests are recorded as customer support tickets and are managed by dedicated support teams.

As organizations continue to grow, the volume of customer support tickets also increases significantly. Managing and analyzing such large volumes of operational data manually becomes difficult, time-consuming, and prone to errors. Raw ticket datasets often contain inconsistencies such as missing information, duplicate records, invalid employee identifiers, unresolved tickets, and data stored in non-standard formats. These inconsistencies make it difficult for management to accurately evaluate employee performance and overall customer support efficiency.

To overcome these challenges, organizations increasingly rely on modern data engineering solutions that automate the process of collecting, cleaning, validating, transforming, and analyzing operational data. Such automated systems ensure that only high-quality and reliable data is used for decision-making, thereby improving operational efficiency and customer satisfaction.

This project focuses on designing a scalable ETL (Extract, Transform, and Load) pipeline capable of processing customer support ticket data using Apache Spark and PySpark within the Databricks environment. The implemented solution demonstrates how raw business data can be transformed into meaningful analytical reports through systematic data validation and business-rule enforcement.

1.2 Introduction to Customer Support Analytics

Customer Support Analytics refers to the process of collecting, processing, and analyzing customer service data to measure operational performance and improve the quality of customer support services. Organizations use analytics to understand how efficiently customer issues are resolved, identify performance gaps among support agents, and optimize resource allocation.

Support ticket analytics enables organizations to answer several important business questions, such as:

- Which team is resolving the highest number of customer issues?
- Which support agents consistently perform well?
- How much time is required to resolve customer tickets?

- Are there any unresolved or low-quality support cases affecting customer satisfaction?
- Which operational teams require additional training or resources?

Traditional reporting methods rely on manually prepared spreadsheets, which become increasingly difficult to manage as the amount of data grows. Automated ETL pipelines overcome these limitations by continuously validating and transforming incoming data while ensuring consistency across all reports.

The analytical reports generated through this project provide management with reliable insights into team performance, helping them make informed decisions to improve customer support operations.

1.3 Problem Statement

The raw customer support ticket data provided for this project contains several data quality issues that prevent direct analytical reporting. These issues include missing Ticket IDs, missing Agent IDs, unresolved support tickets, invalid resolution times, and agents belonging to organizational units outside the defined project scope.

Another challenge is that the resolution time is stored as text rather than numerical values. Since textual values cannot be directly compared mathematically, they must first be converted into minutes before business rules can be applied.

Additionally, operational data is collected across multiple days. If an agent successfully resolves customer issues on the first day and the same agent appears again on the second day, counting both records without appropriate validation would artificially inflate performance metrics. Therefore, a carry-over mechanism is required to ensure that agents who have already contributed valid resolutions on Day 1 are excluded from subsequent calculations.

Without proper validation and transformation, the generated reports would contain inaccurate information, leading to incorrect business decisions and inefficient performance evaluation.

Therefore, an automated ETL pipeline is required to validate, transform, and aggregate customer support ticket data before generating final business reports.

1.4 Need for the Project

Organizations generate vast amounts of operational data every day. Processing such large datasets manually is both inefficient and error-prone. Data inconsistencies can significantly impact reporting accuracy and reduce the reliability of business decisions.

The need for this project arises from the requirement to automate the complete lifecycle of customer support data processing. Instead of relying on manual spreadsheet analysis, the implemented ETL pipeline performs automatic validation, cleansing, transformation, and aggregation of customer support ticket data.

By implementing standardized business rules, the system ensures that only valid and meaningful records are considered during performance evaluation. This improves data quality, minimizes human intervention, and provides management with accurate performance metrics.

Furthermore, the project demonstrates the practical application of distributed data processing using Apache Spark, making it capable of handling larger datasets efficiently. The architecture adopted in this project can easily be extended to enterprise-scale cloud platforms such as Microsoft Azure, enabling organizations to build scalable and reliable data engineering solutions.

1.5 Objectives of the Project

The primary objective of this project is to develop a reliable and scalable ETL pipeline capable of processing customer support ticket data using PySpark and Databricks. The project aims to transform raw operational data into meaningful analytical reports while ensuring high data quality through systematic validation and business-rule enforcement.

The specific objectives of the project are as follows:

- To design a complete ETL pipeline using the Medallion Architecture (Bronze, Silver, and Gold).
- To ingest raw customer support ticket data into the Databricks environment.
- To identify and remove invalid or incomplete records.
- To convert textual resolution times into numerical minute values for analytical processing.
- To apply predefined business rules for data validation and filtering.
- To eliminate duplicate counting through carry-over logic across multiple operational days.
- To generate Team Lead-wise performance reports.
- To generate Agent-wise daily performance reports.
- To export the processed analytical results in CSV format.

- To demonstrate the practical implementation of distributed data processing using Apache Spark.

1.6 Scope of the Project

The scope of this project is limited to processing customer support ticket data generated during two operational days using a simulated dataset within the Databricks environment. The project focuses on implementing an ETL pipeline that performs data ingestion, validation, transformation, filtering, and aggregation according to predefined business requirements.

The project includes the implementation of the Medallion Architecture, where raw data is stored in the Bronze layer, cleaned and enriched in the Silver layer, and aggregated into business-ready reports in the Gold layer.

Although the current implementation uses manually generated datasets stored as Delta tables, the same architecture can easily be extended to process real-time enterprise datasets stored in cloud platforms such as Azure Data Lake Storage Gen2. Additional technologies such as Azure Data Factory, Azure Databricks, Power BI, and real-time streaming frameworks can also be integrated in future versions of the project to support enterprise-scale customer support analytics.

CHAPTER 2

PROBLEM STATEMENT AND BUSINESS OBJECTIVES

2.1 Existing Business Scenario

In modern customer-centric organizations, support teams handle thousands of customer queries every day through multiple communication channels such as email, chat, phone calls, and support portals. Every customer interaction generates a support ticket containing information such as the assigned support agent, ticket status, resolution time, issue category, and team hierarchy. These records are valuable business assets because they help organizations evaluate employee productivity, customer satisfaction, and operational efficiency.

However, operational datasets collected from different teams are often inconsistent and contain incomplete or inaccurate information. Missing identifiers, invalid records, unresolved tickets, duplicate entries, and incorrectly formatted data reduce the reliability of business reports. Consequently, management cannot accurately evaluate support performance or make informed decisions regarding workforce allocation and service improvement.

To overcome these challenges, organizations implement automated data engineering pipelines that systematically process raw operational data before generating analytical reports. Such pipelines ensure data quality, maintain consistency, and enable business leaders to obtain reliable insights from customer support operations.

2.2 Problem Statement

The customer support ticket data provided for this project consists of operational records collected over two consecutive working days. Since the data originates from routine business activities, it contains multiple quality issues that prevent direct analytical processing.

The primary challenges identified in the raw datasets include:

- Missing Ticket IDs, resulting in records that cannot be uniquely identified.
- Missing Agent IDs, making it impossible to associate tickets with responsible employees.
- Resolution Time stored as textual values instead of numerical values.
- Tickets with incomplete or missing resolution time information.
- Tickets that remain in **Open** or **Pending** status.
- Support agents assigned to Team Leads outside the required business scope (TL01–TL08).
- Duplicate performance measurement when the same support agent successfully resolves tickets on multiple operational days.

If these issues are ignored, the generated reports would contain inaccurate statistics and misleading performance metrics. Therefore, an automated ETL solution is necessary to validate, transform, and aggregate the data before analytical reporting.

2.3 Business Objectives

The primary business objective of this project is to develop an automated and scalable data processing pipeline capable of transforming raw customer support ticket data into accurate and meaningful analytical reports.

The project aims to support business decision-making by ensuring that only validated and high-quality records contribute to performance evaluation.

The major business objectives include:

- Improve the quality and consistency of customer support data.
 - Eliminate invalid, incomplete, and duplicate operational records.
 - Standardize resolution time into numerical values suitable for analytical processing.
 - Generate reliable performance metrics for Team Leads and Support Agents.
 - Prevent duplicate counting of agent performance across multiple operational days.
 - Provide management with trustworthy operational insights.
 - Reduce manual effort required for data cleaning and reporting.
 - Demonstrate a scalable ETL architecture suitable for enterprise environments.
-

2.4 Functional Requirements

The ETL pipeline developed in this project is expected to satisfy the following functional requirements:

Data Ingestion

- Read customer support ticket datasets.
- Read support agent profile information.
- Store raw data within the Bronze layer.

Data Validation

- Detect missing Ticket IDs.

- Detect missing Agent IDs.
- Detect missing Resolution Time values.
- Remove invalid records.

Data Transformation

- Convert textual resolution time into numerical minutes.
- Round seconds according to business requirements.
- Standardize ticket status values.
- Join ticket data with employee profile information.

Business Rule Enforcement

- Process only **Resolved** tickets.
- Exclude tickets resolved within 15 minutes or less.
- Include only Team Leads TL01 through TL08.
- Apply carry-over logic to eliminate duplicate agent counting.

Data Aggregation

Generate Team Lead-wise ticket resolution reports.

Generate Agent-wise daily performance reports.

Export reports for business analysis.

2.5 Non-Functional Requirements

Apart from meeting functional objectives, the pipeline should also satisfy several non-functional requirements to ensure efficient execution.

Scalability

The architecture should support processing larger datasets without requiring significant code modifications.

Reliability

The generated reports should consistently produce accurate results after repeated execution.

Maintainability

The modular Bronze-Silver-Gold architecture should simplify future enhancements and maintenance.

Performance

Distributed processing using Apache Spark should reduce execution time for large datasets.

Data Integrity

Business rules should ensure that invalid or duplicate records never appear in final analytical reports.

2.6 Expected Outcomes

Upon successful implementation, the ETL pipeline is expected to produce the following outputs:

- Cleaned and validated customer support ticket data.
- Standardized resolution time expressed in numerical minutes.
- Team Lead Performance Matrix showing the total number of valid resolved tickets handled under each Team Lead.
- Per-Agent Daily Performance Matrix showing individual agent contributions.
- Accurate business reports free from duplicate counting and invalid records.
- CSV files that can be consumed by downstream reporting or dashboarding tools.

These outputs enable management to monitor operational performance, identify high-performing teams, and make data-driven decisions to improve customer support efficiency.

2.7 Success Criteria

The project is considered successful if it meets the following criteria:

- All raw datasets are successfully ingested into the Bronze layer.
- Invalid records are correctly removed during the Silver layer transformations.
- Resolution time is accurately converted into numerical minute values.
- All business rules are correctly enforced.
- Team Lead and Agent performance reports are generated without errors.

CHAPTER 3

TECHNOLOGY STACK

3.1 Introduction

The successful implementation of any data engineering solution depends on selecting technologies that are scalable, efficient, and capable of processing large volumes of data. The Customer Support Ticket Resolution Pipeline was developed using modern big data technologies that are widely adopted in industry for building Extract, Transform, and Load (ETL) pipelines.

The project combines Python programming with Apache Spark's distributed computing capabilities through PySpark and utilizes Databricks as the execution environment. Delta Lake provides reliable storage for intermediate datasets, while Spark SQL is used for structured data manipulation and aggregation. Together, these technologies create a scalable and efficient platform for processing customer support ticket data and generating analytical reports.

3.2 Python

Overview

Python is a high-level, interpreted programming language known for its simplicity, readability, and extensive ecosystem of libraries. It has become one of the most popular programming languages in the fields of data engineering, data science, machine learning, and cloud computing.

Python enables developers to write clean and maintainable code while supporting rapid application development. Its integration with big data frameworks such as Apache Spark makes it an ideal choice for implementing ETL pipelines.

Why Python was Selected

Python was selected for this project because it provides:

- Simple and readable syntax.
- Excellent integration with Apache Spark through PySpark.
- Powerful libraries for data processing.
- Easy implementation of custom functions and validation logic.
- High industry adoption in data engineering projects.

Role in this Project

In this project, Python was used to:

- Generate simulated datasets.

- Define custom functions for parsing resolution time.
- Implement validation logic.
- Apply business rules.
- Control the overall ETL workflow.

Its flexibility made it possible to implement complex transformation logic while maintaining clean and understandable code.

3.3 Apache Spark

Overview

Apache Spark is an open-source distributed data processing framework designed for handling large-scale data efficiently. Unlike traditional data processing systems that repeatedly access disk storage, Spark performs most computations in memory, resulting in significantly faster execution.

Spark supports distributed computing by dividing data across multiple processing nodes, allowing tasks to execute simultaneously. This capability makes Spark suitable for processing millions of records with high performance.

Features of Apache Spark

- Distributed processing architecture.
- In-memory computation.
- Fault-tolerant execution.
- Support for structured and unstructured data.
- High scalability.
- Integration with cloud platforms.

Why Apache Spark was Selected

Apache Spark was selected because it provides:

- Faster execution compared to traditional processing frameworks.
- Efficient handling of large datasets.
- Parallel execution of transformation operations.
- Built-in optimization through Catalyst Optimizer.
- Seamless integration with Databricks.

Role in this Project

Apache Spark serves as the core processing engine for the ETL pipeline. All data ingestion, validation, transformation, filtering, joining, and aggregation operations are executed using Spark's distributed processing capabilities.

3.4 PySpark

Overview

PySpark is the Python API for Apache Spark. It allows developers to write Spark applications using Python while leveraging Spark's distributed execution engine.

PySpark combines the simplicity of Python with the computational power of Apache Spark, enabling developers to process large datasets using familiar programming syntax.

Why PySpark was Selected

PySpark offers several advantages:

- Python-based syntax.
- Distributed data processing.
- DataFrame API for structured data.
- Built-in optimization.
- Easy integration with Spark SQL.

Role in this Project

PySpark was extensively used throughout the project for:

- Reading ticket data.
- Reading agent profile data.
- Data validation.
- Data cleansing.
- Parsing resolution time.
- Joining datasets.
- Filtering records.
- Aggregating reports.

- Exporting final outputs.

The entire ETL pipeline was implemented using PySpark DataFrame operations.

3.5 Databricks

Overview

Databricks is a cloud-based unified analytics platform built on Apache Spark. It provides an interactive environment for developing, executing, and managing big data applications.

Databricks simplifies Spark development by eliminating the need for manual cluster configuration and providing collaborative notebooks where developers can write Python, SQL, Scala, and R code.

Features of Databricks

- Interactive notebooks.
- Integrated Apache Spark.
- Delta Lake support.
- Collaborative development.
- Automatic cluster management.
- Built-in visualization tools.

Why Databricks was Selected

Databricks was selected because it:

- Simplifies Spark development.
- Provides an easy-to-use notebook interface.
- Supports Delta Lake natively.
- Enables rapid experimentation.
- Reduces infrastructure complexity.

Role in this Project

Databricks was used to:

- Develop the ETL notebook.
- Execute PySpark code.

- Store Delta tables.
- Validate intermediate results.
- Display analytical outputs.
- Export the completed notebook.

The notebook environment significantly accelerated project development.

3.6 Delta Lake

Overview

Delta Lake is an open-source storage layer that extends data lakes with additional capabilities such as ACID transactions, schema enforcement, and version control.

Unlike traditional file-based storage, Delta Lake ensures that data remains consistent even when multiple operations are performed simultaneously.

Advantages of Delta Lake

- ACID transactions.
- Schema validation.
- Version history.
- Improved reliability.
- Better query performance.
- Support for scalable analytics.

Why Delta Lake was Selected

Delta Lake was selected because it provides reliable storage for intermediate datasets while maintaining data consistency throughout the ETL process.

Role in this Project

In this project, Delta Lake stores:

- Agent Profiles.
- Day 1 Ticket Logs.
- Day 2 Ticket Logs.

These datasets form the Bronze layer and are subsequently processed into Silver and Gold layers.

3.7 Spark SQL

Overview

Spark SQL is a component of Apache Spark that enables structured data processing using SQL-like operations and DataFrame APIs. It allows users to perform filtering, grouping, aggregation, joins, and analytical queries on structured datasets.

Why Spark SQL was Selected

Spark SQL offers:

- Efficient query execution.
- Optimized DataFrame operations.
- Easy aggregation.
- Seamless integration with PySpark.

Role in this Project

Spark SQL functions were used for:

- Filtering resolved tickets.
- Joining ticket data with agent profiles.
- Grouping records.
- Counting resolved tickets.
- Generating Team Lead Performance Matrix.
- Generating Per-Agent Daily Performance Matrix.

3.8 Summary of Technologies Used

Technology	Purpose in the Project
------------	------------------------

Python	Programming language used to implement ETL logic and business rules
--------	---

Apache Spark	Distributed processing engine for large-scale data processing
--------------	---

PySpark	Python API used for DataFrame transformations and analytics
---------	---

Databricks	Development environment used to execute the ETL pipeline
------------	--

Technology	Purpose in the Project
------------	------------------------

Delta Lake	Storage layer used for Bronze data and reliable data management
------------	---

Spark SQL	Structured querying and aggregation of processed data
-----------	---

3.9 Advantages of the Selected Technology Stack

The chosen technology stack provides several advantages for implementing modern ETL pipelines:

- Supports distributed and scalable data processing.
- Handles large datasets efficiently.
- Simplifies ETL development through PySpark APIs.
- Ensures data reliability with Delta Lake.
- Provides an interactive notebook environment through Databricks.
- Enables rapid development and testing.
- Easily integrates with cloud platforms such as Microsoft Azure.

Overall, the selected technologies complement each other to create a robust, scalable, and efficient data engineering solution suitable for both academic projects and enterprise-level applications.

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 Introduction

The Customer Support Ticket Resolution Pipeline has been designed using a layered architecture to ensure efficient processing, maintainability, and scalability of customer support data. Rather than performing all operations in a single stage, the pipeline separates data processing into multiple logical layers. Each layer is responsible for a specific set of operations and progressively transforms raw operational data into meaningful business insights.

The project follows the **Medallion Architecture**, a widely adopted data engineering design pattern used in enterprise data platforms. The architecture consists of three layers—Bronze, Silver, and Gold—where each layer improves the quality and usability of the data before passing it to the next stage.

By separating data processing into independent layers, the pipeline becomes easier to understand, maintain, debug, and extend. It also ensures that the original raw data remains unchanged, allowing future reprocessing if business requirements change.

4.2 Overall Project Architecture

The overall architecture of the project consists of five major stages:

1. Data Generation and Ingestion
2. Bronze Layer (Raw Data Storage)
3. Silver Layer (Data Cleaning and Transformation)
4. Gold Layer (Business Aggregation)
5. Analytical Report Generation

The following workflow illustrates the complete execution of the project.

CUSTOMER SUPPORT DATA

Agent Profiles Dataset

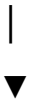
|

|

Day 1 Ticket Logs Dataset

|

|
Day 2 Ticket Logs Dataset



| BRONZE LAYER |
| Raw Delta Tables Stored |



| SILVER LAYER |
| Data Validation |
| Remove Invalid Records |
| Parse Resolution Time |
| Join Agent Profiles |
| Apply Business Rules |



| GOLD LAYER |
| Team Lead Aggregation |
| Agent Aggregation |



	Final CSV Reports	
	Team Lead Performance	
	Agent Performance	

The above architecture demonstrates how raw operational data flows through successive processing stages until meaningful business reports are generated.

4.3 Bronze Layer

The Bronze Layer is the first stage of the Medallion Architecture and serves as the entry point for all incoming data. Its primary purpose is to preserve the raw datasets exactly as they are received, without applying any modifications or business logic.

In this project, three datasets are loaded into the Bronze layer:

- Agent Profiles
- Day 1 Ticket Logs
- Day 2 Ticket Logs

Each dataset is stored as a Delta table within the Databricks environment. Maintaining the raw datasets ensures that the original information remains available for auditing, debugging, and future reprocessing.

The Bronze layer acts as a reliable source of truth for the entire pipeline.

Responsibilities of the Bronze Layer

- Store raw datasets.
 - Preserve original records.
 - Maintain data lineage.
 - Provide input for downstream processing.
 - Enable future reprocessing if required.
-

4.4 Silver Layer

The Silver Layer is responsible for improving the quality of the raw data. All validation, cleansing, enrichment, and business-rule processing takes place at this stage.

This is the most important stage of the ETL pipeline because it converts inconsistent operational data into reliable analytical data.

Several transformations are applied during Silver layer processing.

Step 1 – Data Validation

The pipeline first validates all incoming records to ensure that mandatory information is present.

Records are removed if they contain:

- Missing Ticket ID
- Missing Agent ID
- Missing Resolution Time

Removing incomplete records prevents inaccurate reporting in later stages.

Step 2 – Resolution Time Parsing

The resolution time in the raw dataset is stored as text.

Example:

0h 35m 45s

Such values cannot be used directly for mathematical comparison.

Therefore, a custom PySpark User Defined Function (UDF) converts these values into numerical minutes.

Example:

Original	Converted
-----------------	------------------

0h 35m 45s	36 Minutes
------------	------------

0h 22m 10s	22 Minutes
------------	------------

1h 10m 30s	71 Minutes
------------	------------

The conversion also follows the project requirement of rounding up when seconds are greater than or equal to thirty.

Step 3 – Data Enrichment

After validation, ticket records are joined with the Agent Profiles dataset.

This join adds additional business information such as:

- Agent Name
- Role
- Team Lead ID

As a result, each ticket now contains both operational and organizational information required for reporting.

Step 4 – Business Rule Enforcement

After enrichment, the project applies the required business rules.

The pipeline retains only records that satisfy all of the following conditions:

- Ticket Status must be **Resolved**.
- Resolution Time must be greater than **15 minutes**.
- Team Lead must belong to **TL01–TL08**.
- Invalid records must already be removed.

This filtering ensures that only meaningful business transactions contribute to the final reports.

4.5 Carry-Over Logic

One of the most important features of this project is the implementation of carry-over logic.

Customer support data is collected over multiple days.

If an agent successfully resolves tickets on Day 1 and appears again on Day 2, counting both days without validation would artificially inflate that agent's contribution.

To solve this problem, the pipeline first identifies all successful agents from Day 1.

These agents are then removed from the Day 2 dataset using a **Left Anti Join**.

This approach guarantees that an agent contributes only once to the final business reports, preventing duplicate counting and ensuring fair performance evaluation.

This logic accurately reflects the business requirement specified in the project documentation.

4.6 Gold Layer

The Gold Layer contains the final business-ready datasets generated after all validations and transformations have been completed.

Unlike the Bronze and Silver layers, which focus on data preparation, the Gold layer focuses on business analytics.

Two analytical reports are generated.

Team Lead Performance Matrix

This report displays the total number of valid resolved tickets handled under each Team Lead after all business rules have been applied.

It provides management with an overview of team-level performance.

Per-Agent Daily Performance Matrix

This report summarizes the number of valid tickets resolved by each support agent on each operational day.

It enables management to evaluate individual employee productivity.

4.7 Final Data Flow

The overall execution of the project can be summarized as follows.

Raw Data

↓

Bronze Layer

(Store Raw Delta Tables)

↓

Data Validation

↓

Remove Invalid Records

↓

Resolution Time Parsing



Join with Agent Profiles



Business Rule Filtering



Carry-over Logic



Gold Layer Aggregation



CSV Report Generation



Business Analytics

This workflow demonstrates the complete journey of the customer support ticket data from raw operational records to business-ready analytical reports.

4.8 Advantages of the Adopted Architecture

The layered architecture provides several practical benefits:

- Preserves original raw data for auditing and future processing.
- Separates validation, transformation, and reporting into independent stages.
- Simplifies debugging and maintenance.
- Improves code readability and modularity.
- Supports scalability for larger datasets.
- Enables easy integration with cloud storage and enterprise data platforms.
- Produces reliable, consistent, and business-ready analytical reports.

The use of the Medallion Architecture also aligns the project with industry-standard data engineering practices, making the solution suitable for real-world deployment.

CHAPTER 5

DATASET DESCRIPTION AND BUSINESS RULES

5.1 Introduction

Every data engineering project begins with understanding the structure and quality of the available data. The effectiveness of an ETL pipeline depends on how accurately it processes raw input datasets and transforms them into meaningful business information. In this project, three datasets were used to simulate a real-world customer support environment. These datasets represent employee information and customer support ticket logs collected over two consecutive operational days.

The datasets were created and stored as **Delta Tables** within the Databricks environment to simulate the Bronze layer of the Medallion Architecture. Each dataset serves a specific purpose during the ETL process. The Agent Profiles dataset provides organizational information about support agents, while the Day 1 and Day 2 Ticket Logs contain operational records of customer support activities.

The following sections describe each dataset in detail.

5.2 Agent Profiles Dataset

The Agent Profiles dataset contains the master information of all customer support agents working within the organization. This dataset is used as a reference table during the ETL process to enrich ticket records with organizational information such as the agent's name, role, and reporting Team Lead.

A total of **42 support agents** were included in the dataset. Most agents belong to Team Leads **TL01 to TL08**, while two agents were intentionally assigned to **TL09** and **TL12** to test the implementation of the business rule that restricts processing to a predefined team scope.

Table 5.1: Agent Profiles Dataset Schema

Column Name	Data Type	Description
agent_id	String	Unique identifier assigned to each support agent.
agent_name	String	Name of the customer support agent.
role	String	Designation of the support agent (Junior Support Agent, Support Agent, Senior Support Agent).

Column Name	Data Type	Description
team_lead_id	String	Identifier of the Team Lead supervising the agent.

Purpose of the Dataset

The Agent Profiles dataset is used to:

- Identify each support agent.
- Determine the reporting Team Lead.
- Enrich ticket records with employee details.
- Filter tickets belonging only to the required Team Leads.
- Generate Team Lead and Agent performance reports.

5.3 Day 1 Ticket Logs Dataset

The Day 1 Ticket Logs dataset contains customer support activities recorded during the first operational day. Each record represents a single customer support ticket handled by a support agent.

The dataset intentionally contains both valid and invalid records to simulate real-world operational data. Invalid records include missing Ticket IDs, missing Agent IDs, unresolved tickets, and records with incomplete information. These records allow the ETL pipeline to demonstrate its validation and cleansing capabilities.

A total of **20 ticket records** were created for Day 1.

Table 5.2: Day 1 Ticket Logs Schema

Column Name	Data Type	Description
ticket_id	String	Unique identifier assigned to each customer support ticket.
agent_id	String	Identifier of the support agent handling the ticket.
status	String	Current ticket status (Resolved, Open, Pending).
resolution_time	String	Time taken to resolve the ticket, stored in textual format.

Column Name	Data Type	Description
category	String	Category of customer issue such as Billing, Technical, Returns, Delivery, or Account.

Purpose of the Dataset

The Day 1 Ticket Logs dataset is used to:

- Record customer support activities.
- Measure ticket resolution performance.
- Validate operational records.
- Apply business rules.
- Generate Day 1 performance reports.

5.4 Day 2 Ticket Logs Dataset

The Day 2 Ticket Logs dataset contains ticket records generated during the second operational day. Similar to the Day 1 dataset, it includes both valid and invalid records to evaluate the robustness of the ETL pipeline.

Unlike Day 1, the Day 2 dataset is primarily used to implement the **carry-over logic**, ensuring that agents who have already contributed valid ticket resolutions on Day 1 are not counted again.

A total of **8 ticket records** were included in the Day 2 dataset.

Table 5.3: Day 2 Ticket Logs Schema

Column Name	Data Type	Description
ticket_id	String	Unique identifier assigned to the support ticket.
agent_id	String	Identifier of the assigned support agent.
status	String	Current ticket status.
resolution_time	String	Resolution time stored in text format.
category	String	Customer issue category.

Purpose of the Dataset

The Day 2 Ticket Logs dataset is used to:

- Continue operational tracking.
 - Apply carry-over logic.
 - Prevent duplicate performance counting.
 - Validate multi-day analytical processing.
-

5.5 Summary of Input Datasets

The project uses three primary datasets during execution.

Dataset	Number of Records	Purpose
Agent Profiles	42	Employee master data and Team Lead mapping
Day 1 Ticket Logs	20	Customer support activities for Day 1
Day 2 Ticket Logs	8	Customer support activities for Day 2

These datasets collectively form the **Bronze Layer** of the ETL pipeline.

5.6 Business Rules

Business rules are predefined conditions that determine which records should be included in the analytical reports. These rules ensure that only accurate, complete, and relevant information contributes to the final performance metrics.

The implementation of these rules guarantees that the generated reports accurately reflect organizational performance while eliminating invalid or misleading records.

Rule 1: Remove Records with Missing Ticket ID

Every support ticket must have a unique identifier. Records with missing Ticket IDs cannot be uniquely identified and therefore cannot be reliably tracked.

Such records are removed during the Silver layer validation process.

Example

Ticket ID Result

TKT00015 Accepted

(Blank) Rejected

Rule 2: Remove Records with Missing Agent ID

Each customer support ticket must be associated with a valid support agent.

If the Agent ID is missing, it becomes impossible to determine responsibility for the ticket.

Such records are discarded before further processing.

Rule 3: Remove Records with Missing Resolution Time

Resolution Time is one of the most important attributes used during analytical processing.

Records without resolution time cannot be evaluated and are therefore removed.

Rule 4: Convert Resolution Time into Minutes

The raw dataset stores Resolution Time in textual format.

Example:

0h 35m 45s

Since mathematical operations cannot be performed directly on text values, the pipeline converts the resolution time into numerical minutes.

Examples:

Original Value Converted Value

0h 22m 10s 22 Minutes

0h 35m 45s 36 Minutes

1h 10m 30s 71 Minutes

The conversion follows the business rule that seconds greater than or equal to **30** are rounded to the next minute.

Rule 5: Consider Only Resolved Tickets

Only customer issues that have been successfully resolved are considered for performance evaluation.

Tickets with status **Open** or **Pending** are excluded because they do not represent completed work.

Accepted Status

- Resolved

Rejected Status

- Open
 - Pending
-

Rule 6: Resolution Time Must Be Greater Than 15 Minutes

To avoid counting very short interactions that may not represent complete support cases, the project includes only tickets whose resolution time is **greater than 15 minutes**.

This business rule ensures consistency in performance measurement.

Rule 7: Team Lead Scope Restriction

The project focuses only on support teams managed by Team Leads **TL01 through TL08**.

Agents reporting to **TL09** and **TL12** are intentionally excluded from the final reports.

This rule demonstrates how organizational scope restrictions can be implemented within an ETL pipeline.

Rule 8: Carry-Over Logic

One of the most important business requirements of this project is the implementation of carry-over logic.

If a support agent has already contributed valid ticket resolutions on Day 1, the same agent should not be counted again on Day 2.

The pipeline identifies successful Day 1 agents and performs a **Left Anti Join** with the Day 2 dataset to eliminate duplicate contributions.

This ensures that every support agent contributes only once to the final analytical reports.

5.7 Business Rule Execution Sequence

The ETL pipeline executes the business rules in the following order:

Raw Ticket Data

|



Validate Mandatory Fields

|



Remove Invalid Records

|



Parse Resolution Time

|



Join with Agent Profiles

|



Filter Resolved Tickets

|



Apply Resolution Time Filter

|



Apply Team Lead Scope

|



Apply Carry-Over Logic



Generate Final Reports

5.8 Chapter Summary

This chapter presented a detailed description of the datasets used in the project and explained the business rules that govern the ETL pipeline. Each dataset plays a specific role within the Medallion Architecture, while the business rules ensure that only valid, high-quality records contribute to the final analytical reports. By systematically validating, transforming, and filtering the data, the pipeline produces accurate and reliable performance metrics that can support informed business decision-making.

CONCLUSION

The **Customer Support Ticket Resolution Pipeline** project successfully achieved its objective of developing a reliable and scalable ETL (Extract, Transform, and Load) pipeline for processing customer support ticket data using **PySpark** on the **Databricks** platform. The project demonstrated how raw operational data containing inconsistencies and incomplete records can be transformed into accurate, business-ready information through systematic validation, cleansing, and data transformation techniques.

The pipeline was implemented using the **Medallion Architecture**, consisting of Bronze, Silver, and Gold layers, which provided a structured approach to data processing. The Bronze layer stored the raw datasets, the Silver layer performed data validation and transformation, and the Gold layer generated aggregated reports for business analysis. This layered architecture improved data quality, simplified processing, and ensured that the final outputs were reliable and easy to interpret.

Several business rules were successfully implemented during the ETL process, including the removal of records with missing mandatory fields, conversion of textual resolution time into numerical values, filtering of resolved tickets with resolution times greater than fifteen minutes, restriction of analysis to the required Team Leads (TL01–TL08), and implementation of carry-over logic to prevent duplicate counting of agents across multiple operational days. These rules ensured that only valid and meaningful records contributed to the final analytical reports.

The final outputs of the project included the **Team Lead Performance Matrix** and the **Per-Agent Daily Performance Matrix**, which provide valuable insights into customer support operations. These reports enable management to evaluate individual and team performance, monitor operational efficiency, and support data-driven decision-making. The successful generation of these reports demonstrates the effectiveness of the implemented ETL pipeline.

From a learning perspective, this project provided practical exposure to modern data engineering concepts such as distributed data processing, Delta Lake, DataFrame transformations, joins, aggregations, and business-rule implementation using PySpark. It also enhanced my understanding of Databricks as a cloud-based analytics platform and strengthened my ability to design structured and scalable data pipelines.

In conclusion, this project successfully met all its intended objectives while demonstrating the practical application of data engineering principles in a real-world business scenario. The developed pipeline provides a strong foundation for future enhancements and can be extended to process enterprise-scale customer support data using cloud technologies such as Microsoft Azure, Azure Databricks, Azure Data Factory, and Power BI. Overall, the project has been a valuable learning experience that has improved both my technical knowledge and practical problem-solving skills in the field of data engineering.